SEMANA 16 — INTELIGENCIA ARTIFICIAL EN LA NUBE III

IA en la Nube III: Capacidades y Repaso Unidad 3

Parte 1 de 4 — Modelos de Razonamiento Profundo e Inferencia

Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) — Ingeniería de Sistemas
Semana 16 — Periodo 2026-2 — Barranquilla, Colombia

Parte 1 de 4 — Modelos de Razonamiento Profundo e Inferencia

EVOLUCIÓN DE PARADIGMA

Del Reconocimiento de Patrones al Razonamiento

De respuestas intuitivas inmediatas a procesos deliberativos de deducción lógica paso a paso

Sistema 1: Reconocimiento Rápido

Modo Intuitivo
Emisión inmediata del próximo token más probable

Vulnerabilidad
Alucinaciones en problemas matemáticos y de lógica

Modelos Típicos
LLMs autorregresivos (Claude Sonnet 5.5, GPT-6.1 Sol)

Sistema 2: Razonamiento Deliberativo

Modo Reflexivo
Descompone el problema y evalúa caminos lógicos

Auto-Corrección
Detecta contradicciones internas y retrocede

Modelos Emergentes
Modelos o1, o3-mini y DeepSeek-R1

Fuente: OpenAI — Learning to Reason with LLMs (Research Report 2024)

Escalado del Cómputo en Tiempo de Prueba

Invertir más tokens durante la inferencia incrementa la precisión sin agrandar el modelo

Escalamiento Previo (Preentrenamiento)

Leyes de Chinchilla
Más parámetros y más tokens de entrenamiento

Retorno Decreciente
Costos multimillonarios para mejoras marginales

Límite de Hardware
Modelos de más de 400B saturan centros de datos

Nuevo Paradigma: Test-Time Compute

Escalamiento en Inferencia
El modelo piensa durante más tiempo por consulta

Eficiencia en Parámetros
Un modelo de 32B con razonamiento supera a uno de 405B

Consecuencia de Nube
El costo se traslada del entrenamiento al servicio de API

Fuente: Snell et al. — Scaling LLM Test-Time Compute (UC Berkeley & Google DeepMind 2024)

MECANISMO INTERNO

Cadena de Pensamiento Oculta (Hidden CoT)

Generación de trazas de razonamiento no visibles para el usuario final mediante etiquetas de control

1. Prompt de Entrada

Problema Complejo
Requisitos de arquitectura y red

Sin Instrucción CoT
No requiere 'piensa paso a paso'

Comportamiento Nativo
El modelo decide cuándo reflexionar

2. Bloque <think>

Tokens de Pensamiento
Genera cientos a miles de tokens

Exploración y Descarte
Prueba casos límite y errores

Etiqueta de Cierre
Termina con </think>

3. Respuesta Visible

Síntesis Ejecutiva
Respuesta pulida y estructurada

Alta Consistencia
Sin divagaciones ni errores

Facturación en Nube
Los tokens <think> se cobran

Fuente: DeepSeek-R1 Technical Report & OpenAI Reasoning Architecture (2025)

Árboles de Pensamiento y Backtracking

Exploración sistemática de caminos alternativos y retroceso ante callejones sin salida lógicos

Decodificación Greedy Clásica

Ruta Única Unidireccional
Avanza token por token sin mirar atrás

Efecto Bola de Nieve
Un error inicial arruina todo el razonamiento posterior

Incapacidad de Corrección
El modelo persiste en el error justificándolo

Árbol de Pensamiento (Tree of Thoughts)

Ramificación de Hipótesis
Explora múltiples líneas de deducción en paralelo

Evaluación Heurística
Puntúa la viabilidad de cada rama intermedia

Backtracking Explícito
Poda ramas inválidas y retoma el camino correcto

Fuente: Yao et al. — Tree of Thoughts: Deliberate Problem Solving with Large Language Models (2024)

Entrenamiento sin Supervisión Humana: GRPO

Optimización por políticas de grupo con recompensas basadas en reglas verificables

1. Muestreo de Grupo

Generación Múltiple
Genera N respuestas (ej. N=8)

Diversidad de Trazas
Diferentes caminos deductivos

Cero Humanos en Bucle
No requiere anotadores humanos

2. Función de Recompensa

Verificación Binaria
Compilación y cálculo exacto

Recompensa de Formato
Premia el uso estricto de <think>

Comparativa Relativa
Normaliza ventajas dentro del grupo

3. Actualización de Pesos

Refuerzo de Trazas Válidas
Refuerza caminos óptimos

Sin Modelo Crítico
Ahorra 50% de VRAM en training

Aparición de Auto-Reflexión
El modelo aprende solo a verificar

Fuente: Shao et al. — DeepSeekMath & DeepSeek-R1: Group Relative Policy Optimization (2025)

Verificación Determinista y Recompensas

Sustitución de juicios humanos subjetivos por compiladores y evaluadores formales de software

Modelos de Recompensa Clásicos (RM)

Preferencia Humana Subjetiva
Anotadores juzgan qué respuesta 'suena mejor'

Reward Hacking
Genera textos persuasivos pero falsos

Cuello de Botella
Costoso, lento y no escala a problemas difíciles

Verificadores Deterministas de Reglas

Entornos de Ejecución Reales
Pruebas unitarias, linters y motores formales

Recompensa Binaria Inmutable
+1 si el test pasa en sandbox; 0 si falla o da error

Alineación Rigurosa
Premia la corrección estricta, no la retórica

Fuente: Lambert et al. — Reward Modeling and Rule-Based Verification in LLMs (HuggingFace 2024)

Anatomía de los Tokens de Razonamiento

Diferenciación estricta entre tokens de pensamiento (ocultos) y tokens de respuesta (visibles)

Tokens de Pensamiento (Thinking Tokens)

Naturaleza Operativa
Monólogo interior generado dentro de <think>

Volumen en Producción
Frecuentemente 3x a 10x mayor que la respuesta visible

Régimen Tarifario
Se facturan a tarifa completa de token de salida

Tokens de Salida Visibles (Output Tokens)

Naturaleza Operativa
Texto final emitido al usuario tras </think>

Volumen Típico
Respuesta concisa, diagramas o código depurado

Impacto en FinOps
Total Facturado = Tokens Entrada + Thinking + Visibles

Fuente: Amazon Bedrock API Documentation — Reasoning Tokens Breakdown (2025)

Costos y Retardos en Entornos de Producción

Trade-off directo: mayor calidad cognitiva a cambio de mayor latencia TTFT y costo por llamada

Modelo Convencional (ej. Sonnet 3.5)

Latencia TTFT
500 ms a 1.2 segundos (inmediata)

Tokens Generados
~400 tokens de respuesta directa

Costo Estimado por Llamada
~$0.008 USD (menos de 1 centavo)

Modelo con Razonamiento (ej. o1 / R1)

Latencia TTFT
8 a 45 segundos (tiempo de reflexión)

Tokens Generados
4.500 tokens (4.100 think + 400 respuesta)

Costo Estimado por Llamada
~$0.070 USD (~9x a 10x mayor costo)

Fuente: AWS Bedrock Performance Benchmarks & Real-World Pricing Analysis (2025)

Control de Presupuesto de Razonamiento

Parámetros de API para regular la profundidad del pensamiento según el SLA y el costo

1. Presupuesto Bajo (Low)

Límite de Tokens
budget_tokens: 1.024 tokens

Perfil de Latencia
Latencia reducida (< 5 seg)

Uso Recomendado
Consultas interactivas moderadas

2. Presupuesto Medio (Medium)

Límite de Tokens
budget_tokens: 4.096 tokens

Perfil de Latencia
Latencia moderada (10 a 20 seg)

Uso Recomendado
Refactorización y revisión de PRs

3. Presupuesto Alto (High)

Límite de Tokens
budget_tokens: 16.384+ tokens

Perfil de Latencia
Ejecución asíncrona (> 45 seg)

Uso Recomendado
Diagnóstico crítico y demostraciones

Fuente: Anthropic API — Extended Thinking & Reasoning Budget Configuration (2025)

Auto-Corrección y Debate Multi-Agente

Enfrentamiento dialéctico entre agentes especializados para erradicar sesgos y errores lógicos

1. Agente Proponente

Generación Inicial
Propone diseño o solución de código

Razonamiento Base
Justifica las decisiones adoptadas

Sesgo Típico
Puntos ciegos no anticipados

2. Agente Crítico (Red Team)

Búsqueda de Fallos
Revisa fallos y condiciones carrera

Desafío Dialéctico
Exige pruebas y contraejemplos

Rol Antagónico
Incentivado a encontrar errores

3. Agente Juez (Consenso)

Evaluación Imparcial
Evalúa argumentos del debate

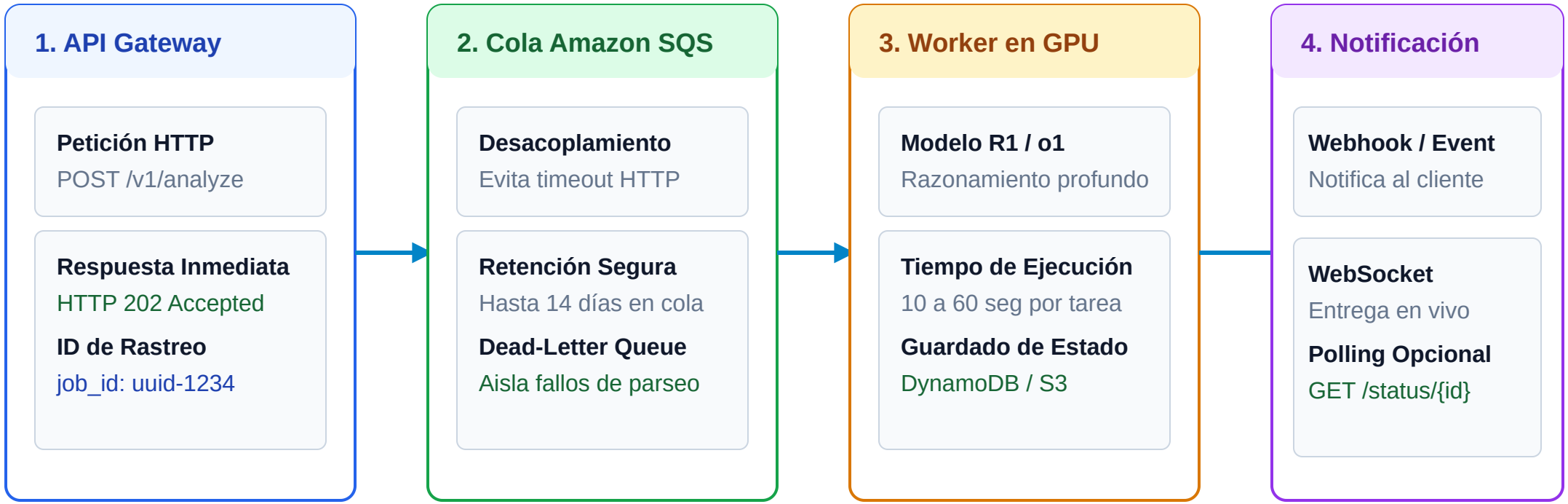
Refinamiento Final
Sintetiza la solución blindada

Resultado Robusto
Tasa de alucinación cercana a cero

Fuente: Du et al. — Improving Factuality and Reasoning in Language Models through Multiagent Debate (2024)

Arquitectura de Servicio para Modelos de Razonamiento

Patrones asíncronos y desacoplados con colas SQS y Webhooks para absorber largas esperas



Fuente: AWS Architecture Center — Asynchronous Generative AI Pattern (2025)

CRITERIOS DE SELECCIÓN

¿Cuándo Usar Razonamiento Profundo?

Matriz de selección: alta complejidad lógica justifica la latencia; tareas rutinarias exigen inmediatez

Usar Razonamiento Profundo (o1 / R1)

Demostraciones y Lógica Formal
Verificación de algoritmos y optimización matemática

Diagnóstico de Fallos Complejos
Análisis cruzado de logs, volcados de memoria y red

Flujos de Trabajo Asíncronos
Revisión nocturna de código y auditoría de seguridad

Evitar y Usar LLMs Estándar (Nova / Claude)

Chat Interactivo y Atención
El usuario no tolerará esperas de 30 segundos

Extracción y Resumen de Textos
RAG tradicional sobre documentos corporativos

Sensibilidad Presupuestal
Millones de consultas simples al día (MaaS ligero)

Fuente: Guía de Selección de Modelos Cognitivos — Universidad de la Costa (2026)

Caso Ilustrativo: Diagnóstico en Nube Privada

Caso ilustrativo de arquitectura, no es una empresa real — Resolución de fallos en OpenStack

Incidencia: Caída de Nodos de Cómputo

Síntoma Complejo
VMs pierden conectividad y Nova Scheduler falla

Datos Recopilados
500 MB de logs de Galera, RabbitMQ y Neutron OVN

LLM Convencional
Sugiere reiniciar Nova pero no detecta quórum de BD

Diagnóstico con Modelo de Razonamiento

Análisis Causal en <think>
Cruza marcas de tiempo entre RabbitMQ y Galera

Identificación de Causa Raíz
Pérdida de quórum por split-brain en red de control

Plan de Remediación
Secuencia ordenada: recuperar nodo primario y OVN

Fuente: Metodología de Diagnóstico de Infraestructura — Laboratorio CUC (2026)

CHECKLIST DE ARQUITECTURA

Principios Clave de Razonamiento en la Nube

Cuatro directrices fundamentales para integrar modelos cognitivos en producción

1. Flujo Asíncrono

Desacoplamiento

Diseñar con colas SQS y WebSockets para absorber tiempos de espera de reflexión

Evitar timeouts

2. Control FinOps

Presupuesto Tokens

Configurar topes de budget_tokens para contener costos de los tokens de think

Límites estrictos

3. Verificación

Pruebas Reales

Combinar deducción con ejecutores de test y compiladores en sandboxes aislados

Cero alucinación

4. Ruteo Híbrido

Gateway de Decisión

Despachar solo el 20 % de tráfico complejo a modelos de razonamiento y 80 % a MaaS ligero

Máximo ROI

SEMANA 16 — INTELIGENCIA ARTIFICIAL EN LA NUBE III

Programación Asistida y Micro-Sandboxing

Parte 2 de 4 — Generación de Código, Evaluación en SWE-bench y Aislamiento en Nube

Ing. Rodolfo Cañas Cervantes

Universidad de la Costa (CUC) — Ingeniería de Sistemas

Semana 16 — Periodo 2026-2 — Barranquilla, Colombia

Parte 2 de 4 — Generación de Código, Evaluación en SWE-bench y Aislamiento en Nube

CAPACIDADES DE DESARROLLO

Modelos Especializados en Código

Entrenamiento con sintaxis formal, pruebas unitarias y repositorios versionados

Modelos Fundacionales

Claude Sonnet 5.5
Líder en SWE-bench Verified

OpenAI o1 / GPT-6.1 Sol
Razonamiento algorítmico formal

Amazon Nova Pro
Generación multilingüe en Bedrock

Modelos Abiertos de Código

DeepSeek Coder V2
Arquitectura MoE para 338 lenguajes

Qwen 2.5 Coder / StarCoder
Auto-alojamiento empresarial local

Qwen 2.5 Coder
Alto rendimiento en 7B y 32B

Habilidades Técnicas

Edición en Múltiples Archivos
Modificaciones consistentes en git

Resolución de Tests
Ejecución de pytest y corrección

Generación de Pruebas
Cobertura de regresión sintética

Fuente: Anthropic / Meta AI — Code Generation Technical Reports (2024)

Evolución de Benchmarks: De Funciones a Repositorios

De acertijos sintéticos aislados a resolución de incidencias en proyectos reales de GitHub

HumanEval / MBPP (2021-2023)

Alcance de Entrada
Una sola función aislada con firma y docstring

Entorno de Ejecución
Sin dependencias externas ni sistema de archivos

Saturación de Métrica
Modelos modernos superan el 90% (benchmark agotado)

SWE-bench Verified (2024-2026)

Alcance de Entrada
Incidencia real en GitHub con miles de líneas de contexto

Entorno de Ejecución
Contenedor completo con entorno virtual y suite de tests

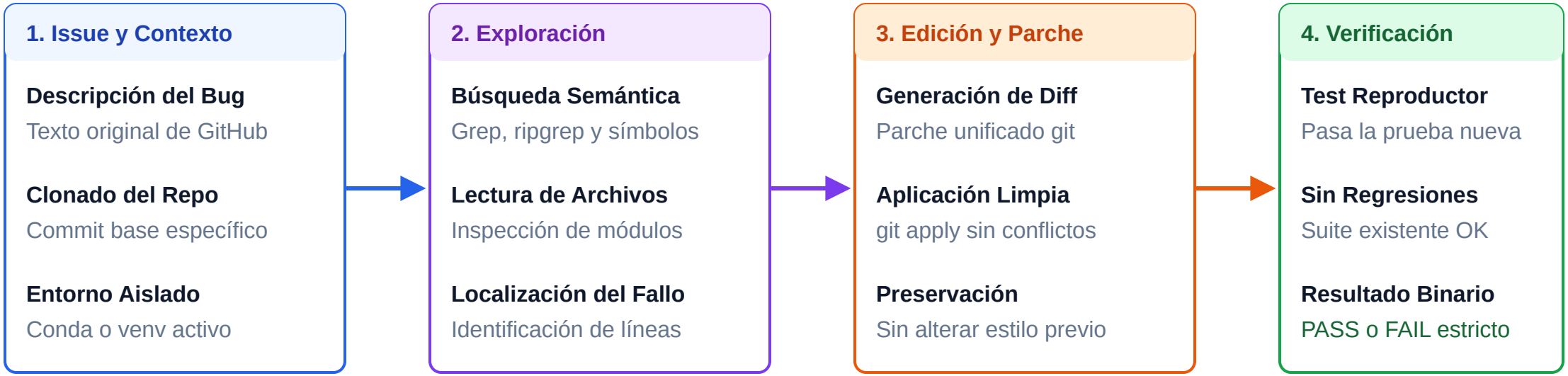
Tasa de Aprobación Real
Agentes avanzados resuelven entre 40% y 55% de tareas

Fuente: Princeton NLP — SWE-bench: Can Language Models Resolve Real-World GitHub Issues? (2024)

MECANISMO DE EVALUACIÓN

Ciclo de Resolución en SWE-bench

Flujo de trabajo autónomo desde la recepción del issue hasta la validación de tests



Fuente: Princeton NLP — SWE-bench Evaluation Harness Architecture (2024)

Indexación Semántica y Árboles Sintácticos

De búsqueda ingenua de texto a navegación por grafo de símbolos y dependencias

Búsqueda Textual Simple

Mecanismo Grep
Coincidencia de cadenas crudas

Limitación Principal
Falsos positivos en comentarios

Sin Contexto
Ignora ámbito y herencia

Parser Tree-sitter (AST)

Árbol de Sintaxis
Identifica clases, funciones y tipos

Segmentación Limpia
Chunking a nivel de método

Filtro Gramatical
Elimina ruido y código irrelevante

Grafo de Dependencias (LSIF)

Ruta de Definición
Navegación 'Go to Definition'

Grafo de Llamadas
Mapeo de invocadores y alcance

Inyección Eficiente
Símbolos relevantes en prompt

Fuente: GitHub Next / Tree-sitter — Semantic Code Search Architecture (2024)

EFICIENCIA EN VENTANA

Edición Quirúrgica vs Reescritura Completa

Minimizar tokens generados y evitar corrupción de archivos grandes

Reescritura Completa del Archivo

Consumo de Tokens
Emisión de miles de líneas innecesarias

Latencia Elevada
Tiempo de espera lineal con el tamaño del archivo

Riesgo de Omisión
El modelo trunca código existente con comentarios

Bloques SEARCH / REPLACE (Quirúrgico)

Consumo Mínimo
Solo emite las 5 a 10 líneas modificadas

Baja Latencia
Respuesta casi instantánea en segundos

Integridad Total
Validación por coincidencia exacta antes de aplicar

Fuente: Aider.chat / Anthropic — Code Editing Benchmarks and Formats (2024)

El Riesgo de Código No Confiable en la Nube

Vectores de ataque al permitir que modelos ejecuten comandos o scripts en servidores

Escape y Destrucción

Borrado Accidental
Ejecución de rm -rf o DROP TABLE

Contenedores Compartidos
Kernel compartido expuesto

Escalamiento Local
Abuso de permisos sudo en el host

Exfiltración de Secretos

Lectura de Variables
Acceso indebido a .env o tokens

Peticiones Clandestinas
Exfiltración a servidores web

SSRF en Nube
Ataque al servicio 169.254

Agotamiento de Cómputo

Bucles Infinitos
Consumo 100% CPU en nodo

Fugas de Memoria
OOM Killer derriba procesos vecinos

Fork Bombs
Saturación de procesos del SO

Fuente: OWASP — Top 10 for Large Language Model Applications (2025)

Aislamiento: Contenedores vs MicroVMs

Comparativa entre namespaces de Linux y virtualización por hardware liviana

Contenedores Docker / Podman

Mecanismo de Seguridad
Namespaces, cgroups y seccomp en kernel compartido

Límite de Aislamiento
Vulnerable si ocurre un fallo en el kernel del host

Tiempo de Inicio
Entre 200 ms y 2 segundos por contenedor

MicroVMs (Firecracker / KVM)

Mecanismo de Seguridad
Kernel independiente con virtualización por hardware

Límite de Aislamiento
Fuerte contención multi-inquilino a nivel de CPU

Tiempo de Inicio
Arranque ultrarrápido en menos de 5 a 10 milisegundos

Fuente: AWS Open Source — Firecracker: Lightweight Virtualization for Serverless (2024)

Arquitectura de Firecracker MicroVM

Monitor de máquinas virtuales minimalista escrito en Rust sobre KVM

Pila de Componentes de Firecracker

Espacio de Usuario Anfitrión

Proceso jailer (chroot y seccomp)

API interna sobre socket UNIX

Emulación mínima de virtio-net

Emulación mínima de virtio-block

Código en Rust con memoria segura

Frontera de Virtualización

Módulo /dev/kvm de Linux

Instrucciones Intel VT-x o AMD-V

Tablas de páginas anidadas (EPT)

Sin dispositivos PCI complejos

Superficie de ataque reducida 95%

MicroVM Huésped (Guest)

Kernel Linux limpio y ligero

Filesystem squashfs solo lectura

Copia efímera de trabajo en ramdisk

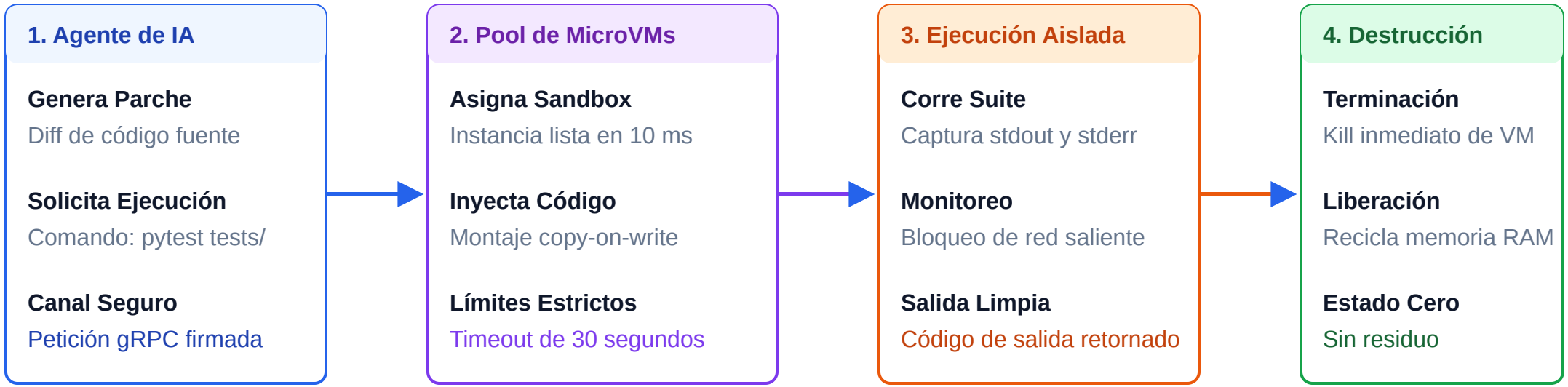
Python, pytest y utilidades de dev

Sin acceso a red externa (aislado)

Fuente: AWS Open Source — Firecracker Design and Implementation (2024)

Ciclo de Ejecución Segura en Nube

Flujo desacoplado: Agente de IA -> Orquestador de Sandboxes -> Resultado de Pruebas



Fuente: AWS Architecture Center — Secure Sandboxing for Generative AI (2024)

Revisión Asistida de Pull Requests en CI/CD

Agentes que analizan diffs, comentan fallas y proponen parches en el pipeline

1. Disparo del Evento

Webhook de GitHub
Evento pull_request.opened

Extracción de Cambios
Git diff y contexto de ramas

Filtro de Ruido
Exclusión de builds y assets

2. Inferencia y Análisis

Chequeo de Reglas
Estándares de arquitectura

Detección de Regresiones
Impacto en contratos de API

Modelos Livianos
Haiku o Nova Lite para costo bajo

3. Retroalimentación

Comentarios en Línea
Sugerencias directamente en el diff

Sugerencia de Commit
Botón de aceptar cambio con 1 clic

Bloqueo de Merge
Fallo del check si hay error crítico

Fuente: GitHub / GitLab — Automated Code Review with AI in Pipelines (2024)

SAST Híbrido: Reglas Deterministas y Modelos

Complementar linters tradicionales con razonamiento semántico para eliminar falsos positivos

SAST Tradicional (Bandit, SonarQube)

Enfoque Basado en Reglas
Búsqueda de patrones rígidos de funciones prohibidas

Alta Tasa de Falsos Positivos
Alerta sobre variables que ya fueron sanitizadas

Ceguera de Negocio
No comprende lógica de autorización entre usuarios

SAST Asistido por LLM (Híbrido)

Análisis del Flujo de Datos
Rastrea la entrada desde el controlador hasta la BD

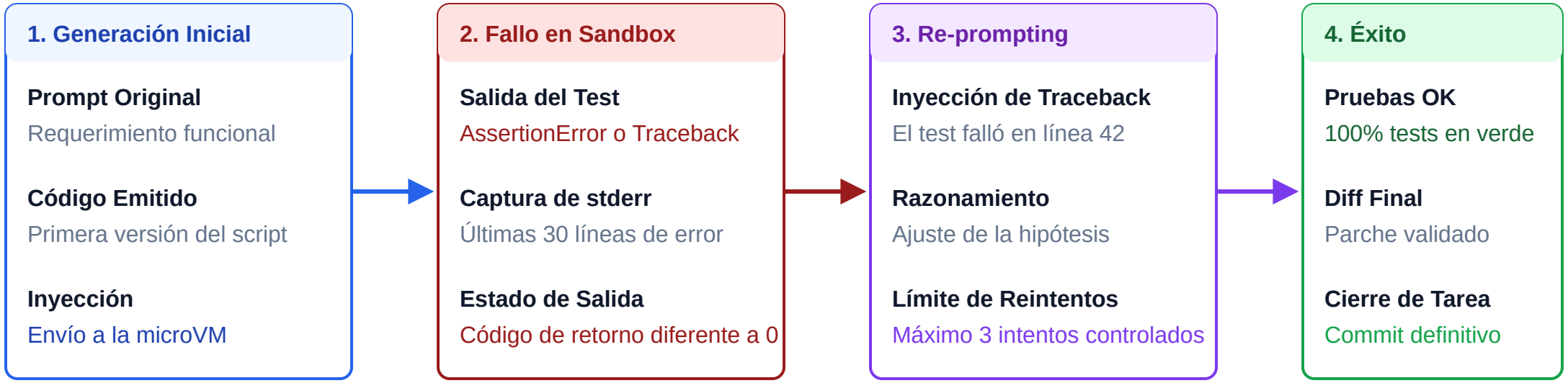
Filtrado Inteligente de Alertas
Descarta avisos si detecta validación previa

Generación del Parche de Remediación
Propone la consulta parametrizada exacta

Fuente: Snyk / SonarQube — AI-Assisted Vulnerability Detection Reports (2024)

Bucle de Retroalimentación de Error

Mecanismo autoreparador: Compilación/Test -> Error -> Re-prompting -> Parche



Fuente: Princeton NLP / SWE-bench Engineering Guide (2024)

CASO ILUSTRATIVO

Caso: Plataforma de Evaluación Automatizada de Código

Caso ilustrativo de arquitectura, no es una empresa real

Entrada y Encolamiento

Recepción de Entregas
API Gateway valida firmas HMAC

Cola de Mensajes
SQS amortigua picos de clase

Cómputo Desacoplado
Workers escalan de forma elástica

Aislamiento por MicroVM

Instancia Dedicada
Una microVM por entrega

Restricción de Recursos
1 vCPU, 512 MB RAM, 10s timeout

Cero Red Externa
Red aislada sin gateway

Calificación y Auditoría

Evaluación de Tests
Puntos por pruebas superadas

Retroalimentación con IA
Explicación didáctica guiada

Almacenamiento Seguro
Reporte firmado en Amazon S3

Fuente: Patrón de referencia de arquitectura de sandboxing para educación superior (2024)

RESUMEN EJECUTIVO

Síntesis de Parte 2: Principios de Código Seguro

Directrices esenciales de arquitectura para integración de modelos de desarrollo

1. Evaluación Rigurosa (SWE-bench)

Pruebas en Repositorios Reales

Descartar benchmarks simples como HumanEval para producción

Medir resolución de incidencias con suites de prueba completas

2. Edición Quirúrgica de Archivos

Bloques Search / Replace y Diffs

Nunca solicitar la reescritura total de archivos extensos

Ahorra hasta 80% de tokens de salida y evita truncamientos

3. Micro-Sandboxing Obligatorio

Fronteras Fuertes con Firecracker

Jamás ejecutar scripts generados en el host de producción

Aislamiento por virtualización KVM sin acceso a red externa

4. Bucle Autoreparador Controlado

Re-prompting con Tracebacks

Inyectar la salida de error del test para corregir el parche

Limitar estrictamente a 3 reintentos para controlar el gasto

Fuente: Guía de Arquitectura de Sistemas de IA en la Nube (2026)

SEMANA 16 — INTELIGENCIA ARTIFICIAL EN LA NUBE III

Sistemas Agénticos y Protocolo MCP

Parte 3 de 4 — Patrón ReAct, Invocación de Herramientas, Memoria y Gobernanza

Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) — Ingeniería de Sistemas
Semana 16 — Periodo 2026-2 — Barranquilla, Colombia

Parte 3 de 4 — Patrón ReAct, Invocación de Herramientas, Memoria y Gobernanza

Del Chatbot Estático al Agente Autónomo

Integración de modelo fundacional, herramientas del entorno, memoria y bucle de decisión

Chatbot Tradicional (Pasivo)

Flujo Monolítico
Entrada de texto del usuario -> Respuesta de texto

Aislamiento Total
Sin acceso a bases de datos ni APIs externas

Límite Temporal
Conocimiento congelado en la fecha de corte

Agente de IA (Activo y Autónomo)

Bucle de Ejecución
Evalúa, selecciona herramientas e itera autónomamente

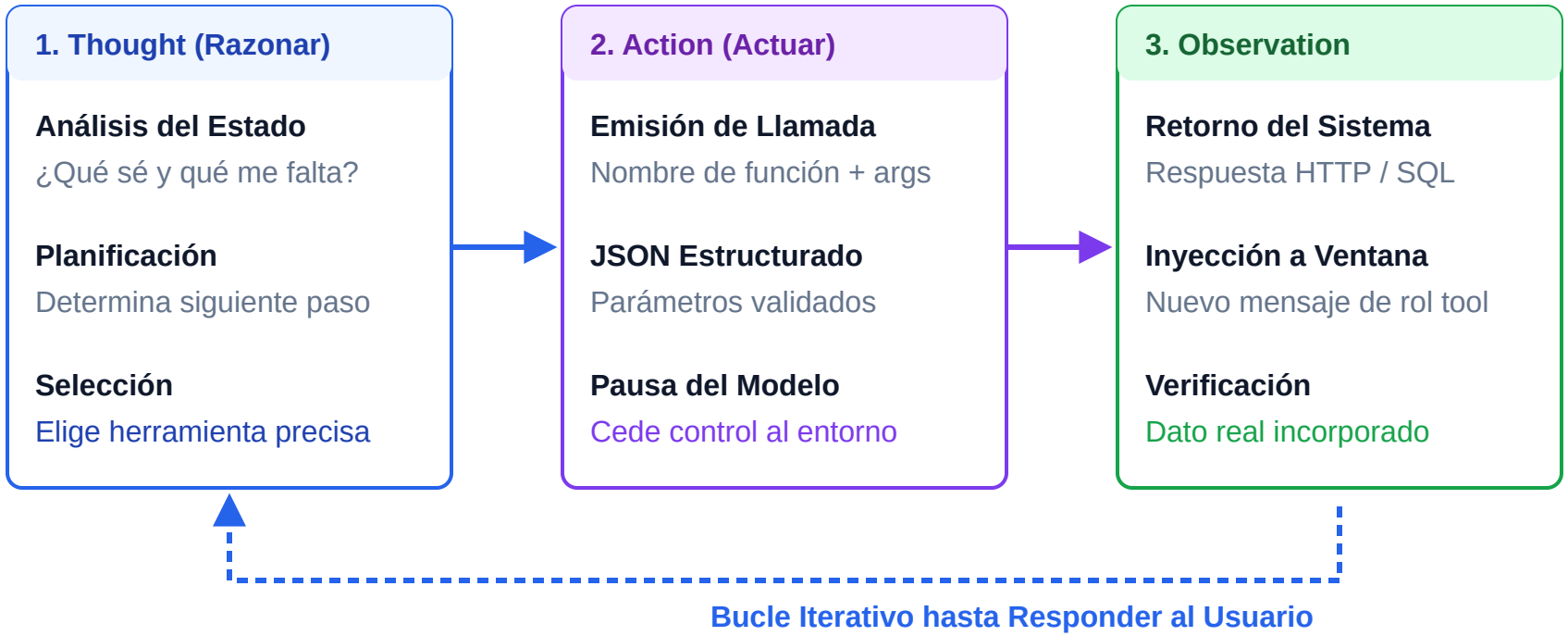
Conectividad Dinámica
Consulta SQL, lee tickets e invoca microservicios

Persistencia y Estado
Mantiene memoria episódica a lo largo de tareas

Fuente: Anthropic — Building Effective Agents (Architectural Guide 2024)

El Patrón ReAct: Thought, Action, Observation

Bucle continuo de deliberación interna, ejecución instrumental y retroalimentación empírica



Fuente: Yao et al. — ReAct: Synergizing Reasoning and Acting in Language Models (ICLR 2023)

Mecanismo Técnico de Function Calling

Cómo el modelo pasa de lenguaje natural a generar estructuras de datos formales

1. Catálogo en el Prompt

Lista de Herramientas
tools: [...] en la petición

Firma Semántica
Nombre claro y descripción

JSON Schema
Tipado de parámetros estrictos

2. Decisión del Modelo

Stop Reason Específico
finish_reason: tool_use

Generación de Argumentos
JSON sintácticamente válido

ID de Invocación
tool_call_id para correlación

3. Backend Ejecutor

Despacho Local
Router llama servicio interno

Control de Permisos
Valida token y rol de usuario

Inyección de Respuesta
Envío del resultado al LLM

Fuente: AWS Bedrock / OpenAI — Function Calling Specification (2024)

Diseño Riguroso de JSON Schema para Herramientas

Descripciones semánticas claras y tipado estricto para evitar alucinaciones en parámetros

Esquema Deficiente (Ambiguo)

Nombre Confuso: 'run_query'

Sin descripción sobre qué tablas o permisos aplican

Parámetros Débiles

query: string sin restricciones ni validación

Consecuencia

Inyecciones SQL, alucinación de sintaxis y fallos en runtime

Esquema Óptimo (Tipado Estricto)

Nombre Preciso: 'consultar_saldo_cuenta'

Descripción clara: 'Retorna saldo y divisa según ID'

Parámetros Acotados con Regex y Enum

cuenta_id: pattern ^CTA-[0-9]{8}$; tipo: enum

Consecuencia

Tasa de éxito superior al 98% sin llamadas espurias

Fuente: JSON Schema Specification (Draft 2020-12) / Anthropic Tool Use Guide (2024)

ESTÁNDAR DE INTEROPERABILIDAD

Model Context Protocol (MCP): El Puerto USB-C de la IA

Estándar abierto para desacoplar modelos de fuentes de datos y servidores de herramientas

Integración Punto a Punto (N x M)

Acoplamiento Fuerte
Cada cliente debe implementar conectores a medida

Deuda Técnica
Cambios en la API rompen todos los clientes existentes

Complejidad O(N * M)
4 clientes x 6 fuentes = 24 conectores independientes

Arquitectura con MCP (N + M)

Protocolo Universal
JSON-RPC 2.0 estandarizado sobre STDIO o SSE

Ecosistema Reutilizable
Un servidor MCP sirve a Claude, Cursor, Antigravity, etc.

Complejidad O(N + M)
4 clientes + 6 servidores = 10 interfaces estándar

Fuente: Anthropic — Model Context Protocol Specification (Nov 2024)

Arquitectura Cliente-Host-Servidor en MCP

Tres primitivas de exposición: Resources (datos), Tools (acciones) y Prompts (plantillas)

MCP Host / Cliente

Aplicación Agente
Claude Desktop / IDE / API

Modelo LLM
Motor de inferencia y decisión

Gestor de Conexiones
Enrutamiento hacia servidores

JSON-RPC

STDIO / SSE

MCP Servers (Proveedores de Contexto)

Resources (Recursos Pasivos)
Lectura de logs, schemas de BD, archivos con URI específica

Tools (Herramientas Activas)
Acciones con efectos secundarios: crear VM, enviar correo, SQL

Prompts (Plantillas Estructuradas)
Flujos de trabajo predefinidos con variables dinámicas

Fuente: Anthropic — Model Context Protocol Architecture (2024)

Mecanismos de Transporte MCP: STDIO vs SSE

De procesos hijos en el host local a microservicios distribuidos en la nube

Transporte Local (STDIO)

Mecanismo de Proceso
El host lanza el servidor como subproceso hijo

Canal de Comunicación
Entrada y salida estándar (stdin / stdout)

Seguridad Implícita
Sin puertos de red expuestos; hereda permisos de usuario

Transporte Remoto en Nube (SSE + HTTP POST)

Mecanismo de Red
Server-Sent Events para streaming + POST para peticiones

Despliegue Distribuido
Contenedor en Kubernetes / ECS detrás de Load Balancer

Seguridad Requerida
Exige mTLS, tokens OAuth 2.0 y autorización por RBAC

Fuente: Model Context Protocol — Transports and Security Specification (2024)

Jerarquía de Memoria en Sistemas Agénticos

De buffers volátiles en contexto a almacenes vectoriales y relacionales persistentes

Memoria de Trabajo

Ubicación
Ventana de contexto del LLM

Latencia
Inmediata (durante la inferencia)

Persistencia
Volátil (desaparece al fin de sesión)

Memoria Episódica

Ubicación
pgvector u OpenSearch

Latencia
Moderada (15 a 50 milisegundos)

Función
Recuerda incidentes y tareas previas

Memoria Semántica

Ubicación
PostgreSQL o DynamoDB

Latencia
Baja (consultas por clave primaria)

Función
Perfiles y reglas de negocio

Fuente: CoALA Architecture — Cognitive Architectures for Language Agents (Sumers et al. 2024)

Compactación y Ventana Deslizante de Contexto

Estrategias para sostener sesiones largas de agente sin agotar la memoria del modelo

Ventana Deslizante Simple (FIFO)

Mecanismo de Descarte
Borra los primeros mensajes al alcanzar el límite

Pérdida de Instrucciones
Olvida el prompt inicial del sistema y restricciones

Deriva Operativa
El agente comienza a comportarse de forma incoherente

Compactación Jerárquica con Resumen

Anclaje del Prompt del Sistema
Las reglas fundamentales jamás se eliminan

Resumen Recursivo
Un LLM liviano condensa los 20 mensajes previos

Preservación de Estado
Conserva hechos críticos reduciendo 80% de tokens

Fuente: Anthropic / OpenAI — Long-Running Agent Sessions Guide (2024)

MITIGACIÓN DE RIESGOS

Acotando el Radio de Impacto (Blast Radius)

Principio de mínimo privilegio, credenciales con alcance limitado y aprobaciones humanas

1. Privilegios Mínimos

Tokens de Solo Lectura
Por defecto sin permisos de escritura

Alcance de Recurso
Solo accede al prefijo de su proyecto

Credenciales Efímeras
AWS STS con expiración en 15 min

2. Human-in-the-Loop

Acciones Destructivas
Eliminar recursos o pagar cuentas

Confirmación Explícita
Pausa y espera aprobación humana

Registro en Auditoría
Quién aprobó y en qué timestamp

3. Rate Limiting y Cuotas

Límite de Invocaciones
Máximo 10 acciones por minuto

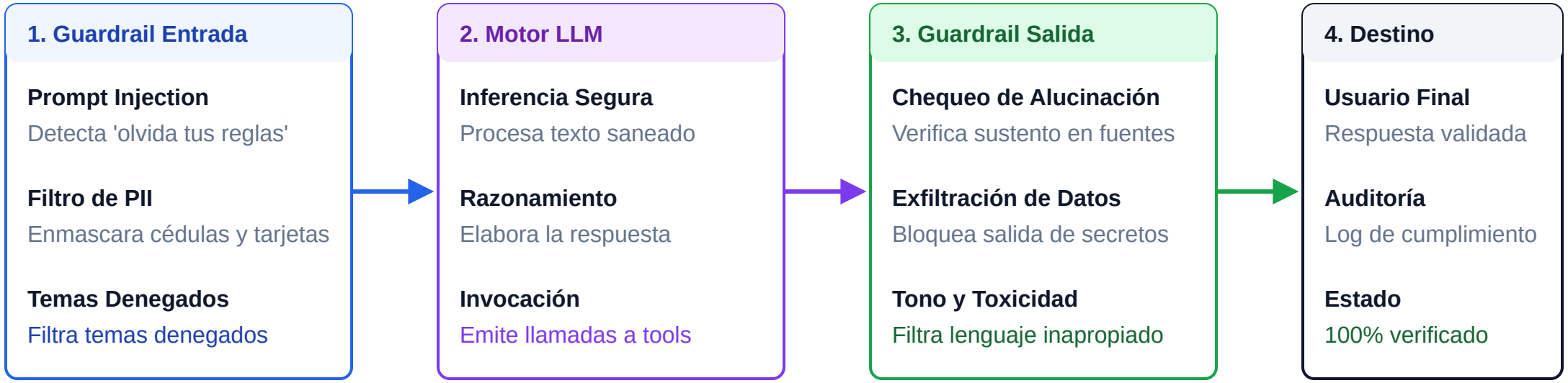
Tope de Gasto en Dólares
Freno al alcanzar el budget

Alertas Inmediatas
Notificación por SNS ante anomalías

Fuente: AWS Security Best Practices for Generative AI Applications (2024)

Guardrails: Filtros de Entrada y Salida

Capas de validación deterministas y semánticas antes y después del modelo



Fuente: AWS Bedrock Guardrails / NeMo Guardrails Documentation (2024)

Patrones de Orquestación Multi-Agente

De agentes solitarios a equipos cooperativos: Supervisor central vs Enrutamiento dinámico

Patrón Supervisor Centralizado

Agente Director
Descompone la tarea y asigna subtareas a especialistas

Control Jerárquico
Los especialistas reportan exclusivamente al supervisor

Ventaja Principal
Flujo predecible, fácil de auditar y depurar

Patrón Enrutamiento por Especialidad (Router)

Clasificador Inicial
Un clasificador rápido dirige el flujo al agente idóneo

Traspaso Directo (Handoff)
El especialista toma el control total de la interacción

Ventaja Principal
Baja latencia y menor consumo de tokens acumulados

Fuente: LangChain / AutoGen — Multi-Agent Systems Architectural Patterns (2024)

CASO ILUSTRATIVO

Caso: Mesa de Ayuda Agéntica con Servidores MCP

Caso ilustrativo de arquitectura, no es una empresa real

Canal de Entrada

Tickets de Usuarios
Recepción por Webhook o Portal

Clasificación Automática
Categoría: Redes / Acceso / BD

Filtro de Seguridad
Enmascara contraseñas y PII

Agente y Conectores MCP

Servidor MCP Jira
Lectura de histórico de tickets

Servidor MCP Active Directory
Verificación de estado de cuenta

Servidor MCP Monitoreo
Consulta estado de VPN y enlaces

Resolución Controlada

Diagnóstico Guiado
Respuesta con pasos específicos

Desbloqueo de Usuario
Requiere token MFA confirmado

Cierre del Ticket
Actualización de SLA y log

Fuente: Patrón de referencia para mesa de ayuda asistida por IA y MCP (2024)

RESUMEN EJECUTIVO

Síntesis de Parte 3: Principios de Sistemas Agénticos

Reglas de oro para construir y desplegar agentes confiables en la nube

1. Patrón ReAct y JSON Schema Estricto

Contratos de Datos Inmutables

Tipar parámetros con expresiones regulares y enumeraciones

El backend valida permisos antes de ejecutar cualquier acción

2. Adopción de Model Context Protocol (MCP)

Desacoplamiento Estándar

Separar datos (Resources), acciones (Tools) y plantillas (Prompts)

Evolución de STDIO local a microservicios distribuidos con SSE

3. Jerarquía y Compactación de Memoria

Gestión de Contexto Sostenible

Memoria de trabajo, vectores y BD

Compactar mediante resúmenes sin descartar el prompt de sistema

4. Blast Radius y Guardrails Estrictos

Seguridad y Defensa en Profundidad

Tokens efímeros, cuotas de gasto y aprobación humana obligatoria

Filtros de prompt injection al entrar y anti-fugas al salir

SEMANA 16 — INTELIGENCIA ARTIFICIAL EN LA NUBE III

Repaso Integral de la Unidad 3

Parte 4 de 4 — Síntesis de Nubes Privadas, Cómputo GPU, Inferencia y Agentes

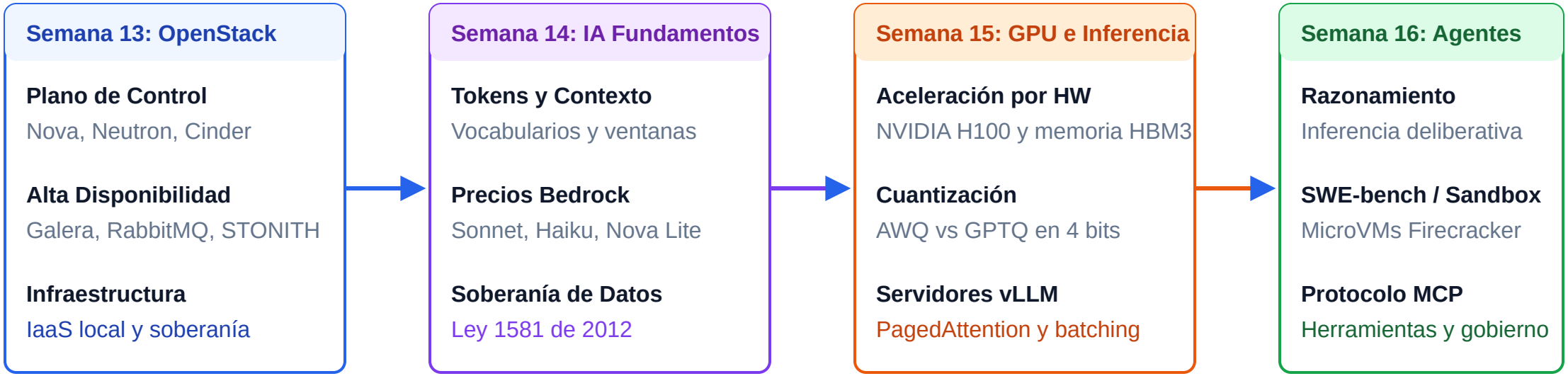
Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) — Ingeniería de Sistemas
Semana 16 — Periodo 2026-2 — Barranquilla, Colombia

Parte 4 de 4 — Síntesis de Nubes Privadas, Cómputo GPU, Inferencia y Agentes

VISIÓN HOLÍSTICA

Mapa de Ruta: De la Infraestructura Base a la IA

Cuatro semanas articuladas: Nube privada, inferencia básica, aceleración y sistemas agénticos



Fuente: Silabo de Arquitectura en la Nube — Universidad de la Costa (2026)

OpenStack: Ciclo de Vida de Creación de una VM

Secuencia coordinada: Keystone -> Nova API -> Glance -> Neutron -> Cinder -> Nova Compute

Secuencia de Interacción entre Servicios OpenStack

1. Autenticación

Keystone

Valida usuario

Emite token Fernet

Catálogo endpoints

Token verificado

2. Entrada Nova

Nova API / Conductor

Verifica cuotas

Registra registro BD

Nova Scheduler

Elige hipervisor

3. Red y Disco

Neutron y Cinder

Neutron reserva IP

Crea puerto virtual

Cinder adjunta LUN

Recursos listos

4. Imagen SO

Glance

Descarga qcow2/raw

Transfiere a nodo

Copia efímera

Imagen en caché

5. Hipervisor

Nova Compute

Invoca libvirt / KVM

Enchufa vNIC a OVN

Arranque del guest

Estado: ACTIVE

Fuente: OpenStack Architecture Guide (2024.1 Caracal)

Redes de Proyecto (Overlay) vs Proveedor (Flat/VLAN)

Aislamiento multi-inquilino con túneles Geneve frente a conectividad física directa

Redes de Proyecto (Overlay Geneve / VXLAN)

Encapsulamiento L2 sobre L3
Paquetes encapsulados en UDP sobre la red de transporte

Autonomía Total del Usuario
El tenant define libremente sus rangos IP y subredes

Salida Externa
Requiere enrutador virtual y Floating IPs para internet

Redes de Proveedor (Flat / VLAN Física)

Conexión Directa al Switch
Mapeo directo a la infraestructura física del datacenter

Sin Sobrecarga de Encapsulado
MTU estándar de 1500 bytes sin cabeceras extra

Gobernanza Centralizada
Solo administradores de red pueden crearlas y asignarlas

Fuente: OpenStack Neutron Documentation — Networking Concepts (2024)

Alta Disponibilidad: Galera, RabbitMQ y STONITH

Tolerancia a fallos en el plano de control: Quórum de base de datos, colas espejo y esgrima de nodos

MariaDB Galera Cluster

Replicación Síncrona
Todos los nodos tienen el estado

Regla de Quórum
Mínimo 3 nodos impares

Prevención Split-Brain
La minoría se desconecta sola

RabbitMQ Clustered

Bus RPC Central
Comunica todos los demonios

Quorum Queues
Basadas en algoritmo Raft

Alta Disponibilidad
Mensajes replicados en 3 nodos

Pacemaker + STONITH

Corosync Heartbeat
Detección de vida en milisegundos

Esgrima (Fencing)
Shoot The Other Node In The Head

IP Virtual Flotante
Failover transparente de VIP

Fuente: OpenStack High Availability Guide (2024.2)

Hardware de Aceleración: GPUs y Memoria HBM3

Superar el cuello de botella de memoria: Ancho de banda extremo e interconexiones de baja latencia

Arquitectura CPU Tradicional

Memoria del Sistema (DDR5)
Ancho de banda moderado: 200 a 300 GB/s

Enfoque de Cómputo
Pocos núcleos ultra-rápidos para flujos secuenciales

Límite en Inferencia LLM
Saturación por transferencia de pesos en cada token

Acelerador GPU (NVIDIA H100 SXM)

Memoria Apilada HBM3
Ancho de banda masivo: 3.35 TB/s (10x superior)

Tensor Cores de 4ª Generación
Aceleración por hardware para tipos FP8 y FP16

Interconexión NVLink 4
900 GB/s bidireccionales entre GPUs del mismo nodo

Fuente: NVIDIA H100 Tensor Core GPU Architecture Whitepaper (2024)

Cuantización: AWQ vs GPTQ en Inferencia

Reducción de memoria a 4 bits protegiendo los pesos salientes del modelo

GPTQ (Compresión Estática por Capas)

Mecanismo Matemático
Ajusta pesos restantes compensando el error cuadrático

Rendimiento en Inferencia
Óptimo en lotes pequeños mono-usuario

Sensibilidad de Dominio
Sensible en código y razonamiento

AWQ (Protección por Activaciones Salientes)

Observación Clave
Solo el 1% de los pesos concentra la precisión crítica

Protección Selectiva
Protege el 1% sensible antes de INT4

Soporte en Servidores
Optimizado nativamente en vLLM y TensorRT-LLM

Fuente: Lin et al. — AWQ: Activation-aware Weight Quantization (MLSys 2024)

vLLM: PagedAttention y Batching Continuo

Eliminación de la fragmentación de memoria y saturación óptima del cómputo GPU

Servidores Tradicionales (Fragmentación)

Asignación Contigua Estática
Reserva memoria para max_context_length por anticipado

Desperdicio Masivo de VRAM
Entre 60% y 80% de la memoria queda bloqueada sin uso

Batching Estático Rígido
Espera a que todas las peticiones del lote terminen

vLLM con PagedAttention

Paginación Virtual del KV Cache
Divide la memoria en páginas no contiguas de 16 tokens

Desperdicio Mínimo (< 4%)
Permite hasta 4 veces más usuarios concurrentes por GPU

Batching Continuo a Nivel de Token
Nuevas peticiones entran inmediatamente al lote activo

Fuente: Kwon et al. — Efficient Memory Management for LLM Serving with PagedAttention (SOSP 2023)

Amazon Bedrock: Catálogo Multimodelo Serverless

Inferencia elástica bajo demanda sin gestión de servidores, clusters ni GPUs físicas

1. Catálogo Unificado

Múltiples Proveedores
Anthropic, Meta, Amazon, Cohere

API Estándar Converse
Un solo código para todo modelo

Model Routing
Enrutamiento inteligente por costo

2. Estructura de Precios (1M)

Nova Micro / Lite
$0.035 / $0.06 in | $0.14 / $0.24 out

Claude Haiku 5.5
$0.10 entrada | $0.50 salida

Claude Sonnet 5.5
$2.00 entrada | $10.00 salida

3. Privacidad y Seguridad

Sin Re-entrenamiento
Datos no entrenan modelos

Aislamiento en VPC
AWS PrivateLink sin transitar internet

Prompt Caching
90% descuento en lectura de caché

Fuente: AWS Bedrock Documentation & Official Pricing (Consultado Noviembre 2026)

Evaluación de TCO: ¿Cuándo Auto-Alojar vs API?

Análisis del punto de equilibrio entre costos fijos de infraestructura y costos variables por token

Servicio Gestionado por API (Bedrock)

Estructura Financiera: OPEX Puro
Costo cero cuando la demanda es cero; pago por token

Carga Operativa Mínima
Cero mantenimiento de CUDA o parches

Cuándo Conviene
Prototipos, tráfico variable o menos de 50M tokens diarios

Auto-Alojamiento Dedicado (EC2 / Local)

Estructura Financiera: Costo Fijo Mensual
Costo de instancias GPU encendidas 24/7 sin importar uso

Alta Carga de Ingeniería
Requiere equipo especializado en MLOps, vLLM y métricas

Cuándo Conviene
Tráfico masivo (>100M/día) o soberanía

Fuente: AWS Cloud Economics — Total Cost of Ownership for LLMs (2024)

Soberanía de Datos y Cumplimiento: Ley 1581

Requisitos legales en Colombia para procesamiento de datos personales en la nube

1. Principio de Finalidad

Autorización Previa
Consentimiento explícito del titular

Uso Específico
Solo para la tarea contratada

Prohibición de Reúso
No entrenar modelos públicos

2. Transferencia Internacional

Países con Estándar Adecuado
Circular Única de la SIC

Contrato de Transmisión
DPA con el proveedor de nube

Región de Despliegue
Elección de zona geográfica

3. Seguridad y Anonimización

Cifrado de Extremo a Extremo
En tránsito (TLS) y reposo (KMS)

Anonimización Previa
Enmascarar datos previos

Pista de Auditoría
Registro inmutable en CloudTrail

Fuente: Superintendencia de Industria y Comercio (SIC) — Guías de Cumplimiento Ley 1581 (2024)

GUÍA DE DECISIÓN

Matriz de Selección: ¿Qué Tecnología Elegir?

Árbol de criterios según latencia, presupuesto, volumen de datos y criticidad de seguridad

Criterios de Elección Tecnológica en la Unidad 3

OpenStack Local

Soberanía Estricta
Datos que no pueden salir del campus

Control de Hardware
Sin costos variables

IaaS bajo tu control

Amazon Bedrock

Velocidad al Mercado
Despliegue inmediato sin comprar servidores

Costo Flexible
Pagas solo por token

Inferencia Serverless

Clúster vLLM + AWQ

Alto Rendimiento
Rendimiento predecible sin límites de cuota

Volumen Masivo
Costo fijo eficiente

Auto-alojamiento top

Agentes con MCP

Automatización Real
Ejecución con guardrail y sandboxing seguro

Ecosistema Abierto
Herramientas modulares

Delegación confiable

Fuente: Guía de Arquitectura de Sistemas de IA en la Nube (2026)

PERFIL PROFESIONAL

El Arquitecto de Soluciones en la Era de la IA

De simple aprovisionador de máquinas virtuales a diseñador de sistemas cognitivos confiables

Arquitecto Clásico

Enfoque Primario
Redes, VPCs y almacenamiento

Métricas Típicas
Uptime, IOPS y utilización de CPU

Límites del Rol
La aplicación era una caja negra

Arquitecto de IA y Nube

Enfoque Primario
Flujos de inferencia y tokens

Métricas Típicas
TTFT y costo por token

Integración Total
Diseña el entorno cognitivo

Competencias Clave

Diseño de Guardrails
Fronteras estrictas de seguridad

Optimización de Costos
Caché, cuantización y enrutamiento

Cumplimiento Ético
Soberanía y protección del usuario

Fuente: AWS Certified Solutions Architect Professional Framework (2026)

CHECKLIST DE ENTREGA

Checklist de Arquitectura: Listos para Producción

Seis preguntas obligatorias antes de poner en marcha un sistema de IA en la nube

1. ¿Está acotado el radio de impacto de las herramientas?

Privilegios mínimos y esquemas tipados

2. ¿Se ejecuta el código en sandboxes aislados?

MicroVMs Firecracker efímeras sin acceso a red externa ni host

3. ¿Existe un tope presupuestal y cuotas por usuario?

Límites de tokens y cortacircuitos

4. ¿Operan guardrails de entrada y salida?

Detección inyección y filtro de PII

5. ¿Se cumple el marco de la Ley 1581 de datos?

Autorización, DPA firmado y cifrado

6. ¿Está justificada la elección entre API y Auto-alojamiento?

TCO validado según volumen y equipo

Fuente: AWS Well-Architected Framework — Generative AI Lens (2024)

Conclusión de la Unidad 3: El Futuro de la Nube

Integración armónica entre infraestructura sólida, modelos deliberativos y gobernanza responsable

Tres Pilares Inmutables de la Arquitectura en la Nube

1. Fundamentos Sólidos

Infraestructura Resiliente

Redes OVN y alta disponibilidad

Almacenamiento y quórum Galera

Hardware GPU y enlaces HBM3

Sin base, ningún LLM escala

2. Eficiencia y Escala

Optimización de Recursos

Cuantización AWQ en 4 bits

PagedAttention y batching en vLLM

Serverless en Amazon Bedrock

Máximo valor por dólar invertido

3. Gobierno y Autonomía

Sistemas Agénticos Seguros

Protocolo abierto MCP

Micro-sandboxes con Firecracker

Cumplimiento estricto Ley 1581

IA confiable y ética al servicio